

Лабораторная работа №13-14. ViewModel и архитектура Android-приложений

Цель работы: Познакомиться с архитектурными компонентами **Android Jetpack**, научиться использовать **ViewModel** для управления состоянием **UI** и сохранения данных при изменении конфигурации устройства (поворот экрана, смена темы и т.д.). Разработать игру с подсчётом очков, которая не теряет данные при повороте экрана.

Теоретическая часть

Что такое Android App Architecture?

Архитектура приложения — это набор правил проектирования, которые определяют структуру вашего приложения. Подобно чертежу дома, архитектура обеспечивает надёжность, гибкость, масштабируемость и удобство поддержки кода на долгие годы.

Зачем это нужно?

- Легко добавлять новые функции
- Код можно тестировать
- Легко находить и исправлять баги
- Несколько разработчиков могут работать над одним проектом

Проблема: что происходит при повороте экрана?

Вспомните из прошлой лабораторной работы: когда вы поворачиваете телефон, **Android полностью уничтожает** вашу **Activity** и создаёт её заново. Это называется **configuration change** (изменение конфигурации).

Что мы теряем:

// Обычные переменные в Composable

```
var score by remember { mutableStateOf(0) }
```

// После поворота экрана score снова будет 0!

Что мы пробовали раньше:

- **rememberSaveable** — работает, но логика хранится прямо в **Composable**
- Сохранение **instance state** — много шаблонного кода

Знакомство с ViewModel

ViewModel — это специальный класс из Android Jetpack, который:

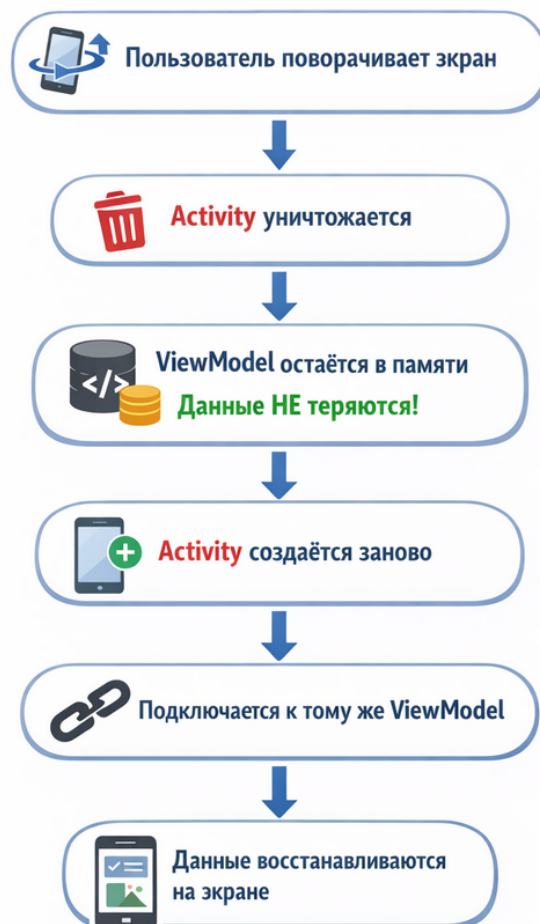
- **Хранит данные** вашего UI отдельно от Composable-функций
- **Сохраняется при configuration changes** (поворот экрана, смену темы)
- **Автоматически очищается** когда приложение закрывается
- **Легко тестируется** (можно писать unit-тесты)

Важно понимать:

- ViewModel живёт **дольше** чем Composable-функции
- ViewModel **НЕ сохраняется при** закрытие процесса (когда Android убивает приложение из-за нехватки памяти)
- ViewModel **не должен** содержать Context или View (это может привести к утечкам памяти)

Как работает ViewModel?

Как работает ViewModel?



Сравнение подходов к сохранению состояния

Подход	Где хранятся данные	Переживает поворот?	Переживает смерть процесса?	Удобство
remember	В Composable	✗ Нет	✗ Нет	Просто, но ненадёжно
rememberSaveable	В Bundle	✓ Да	✓ Да	Ограничен простыми типами
ViewModel	В отдельном классе	✓ Да	✗ Нет	✓ Идеально для UI логики
Database	На диске	✓ Да	✓ Да	Сложно, для постоянных данных

StateFlow: современный способ работы с состоянием

StateFlow — это специальный тип переменной, которая:

- Автоматически **уведомляет** UI об изменениях
- **Thread-safe** (безопасна при работе с несколькими потоками)
- Работает по принципу “**подписки**” (UI подписывается на изменения)

Сравнение с обычными переменными:

// Обычная переменная

```
var score = 0
```

```
score = 10 // UI НЕ обновится автоматически
```

// StateFlow

```
private val _score = MutableStateFlow(0)
```

```
val score: StateFlow<Int> = _score
```

```
_score.value = 10 // UI обновится АВТОМАТИЧЕСКИ!
```

UI State: единое хранилище состояния

Вместо множества отдельных переменных, мы создаём **один data class**, который содержит всё состояние экрана:

// Плохо: много разных переменных

```
var score = 0
```

```
var currentWord = ""
```

```
var isGameOver = false
```

```
var wordCount = 1
```

// Хорошо: всё в одном месте

```
data class GameState(  
    val score: Int = 0,  
    val currentWord: String = "",  
    val isGameOver: Boolean = false,  
    val wordCount: Int = 1  
)
```

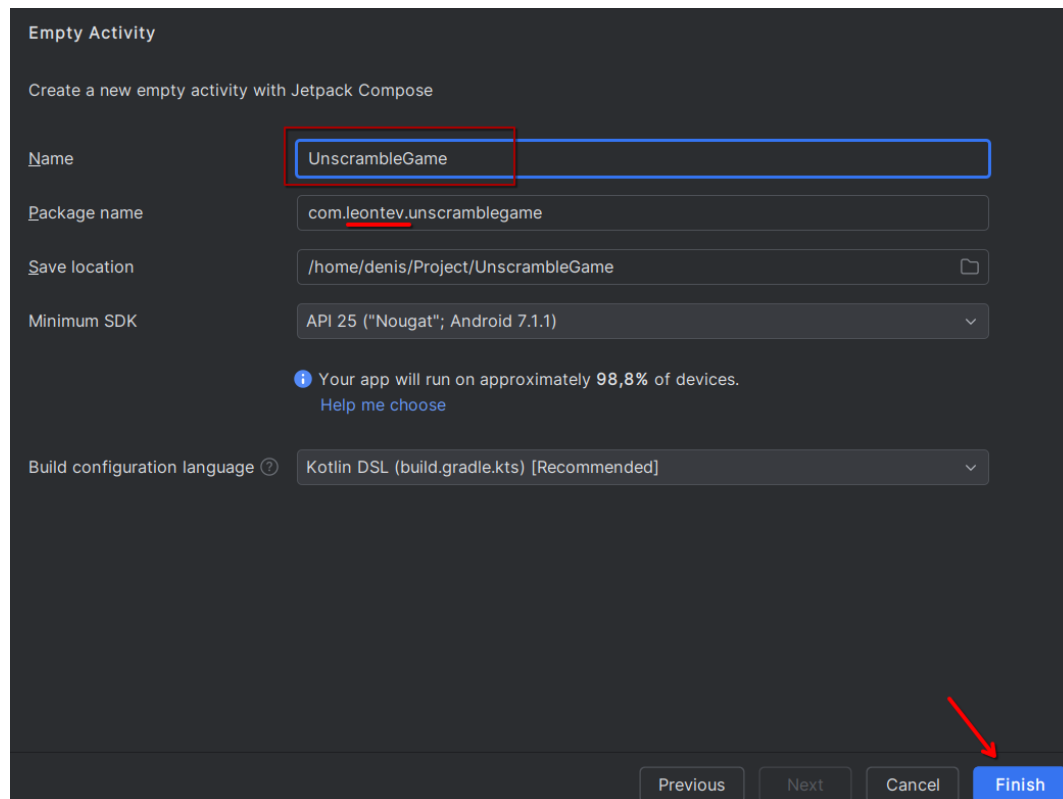
Зачем это нужно?

- Всё состояние в одном месте
- Легко понять, что отображается на экране
- Легко тестировать
- Легко сохранять и восстанавливать

Практическая часть. Подготовка к работе

Создание проекта

1. Откройте **Android Studio**
2. Выберите **File → New → New Project**
3. Выберите **Empty Activity → Next**



4. Укажите параметры:

- **Name:** `UnscrambleGame`
- **Package name:** `com.ваше_фio.unscramblegame`
- **Minimum SDK:** API 25

5. Нажмите **Finish**

Дождитесь загрузки проекта.

Создание Git-репозитория

Перед началом работы создайте Git-репозиторий для отслеживания изменений и защиты от потери данных.

Вариант 1: Через Android Studio (рекомендуется)

1. В Android Studio выберите **VCS → Enable Version Control Integration**
2. Выберите **Git → OK**
3. Выберите **Git → Commit** (или `Ctrl+K`)
4. **Выберите все файлы**, введите сообщение: `"Initial commit"`
5. Нажмите **Commit**

Вариант 2: Через терминал

1. Откройте терминал в Android Studio (**View → Tool Windows → Terminal**)
2. Выполните команды:

```
git init
```

```
git add .
```

```
git commit -m "Initial commit"
```

Создание удалённого репозитория на GitHub

1. Откройте **GitHub** и войдите в аккаунт.
2. Нажмите **New repository** (зелёная кнопка)
3. **Repository name:** `UnscrambleGame_FIO` (где FIO — ваша фамилия)
4. Выберите **Public** (публичный репозиторий)
5. **НЕ** ставьте галочки на "Add README" и ".gitignore"
6. Нажмите **Create repository**
7. Скопируйте URL репозитория

Привязка локального репозитория к удалённому

В терминале Android Studio выполните:

git branch -M main

git remote add origin <URL_репозитория>

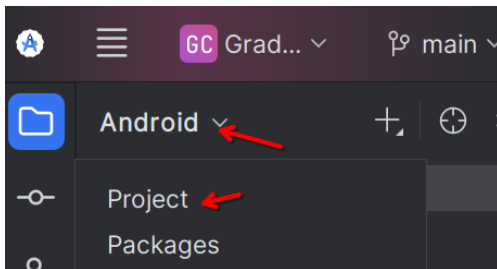
git push -u origin main

* Замените <URL_репозитория> на скопированный URL

При первом **push** вас попросят ввести логин и пароль **GitHub**.

3. Создание папки **img**

1. В Android Studio переключитесь в режим просмотра **Project** (в левом верхнем углу панели Project)



2. Найдите **app**

3. Кликните правой кнопкой на папке **app**

4. Выберите **New** → **Directory**

5. Введите имя: **img**

6. Нажмите **OK**

Полный путь к папке: /app/img/

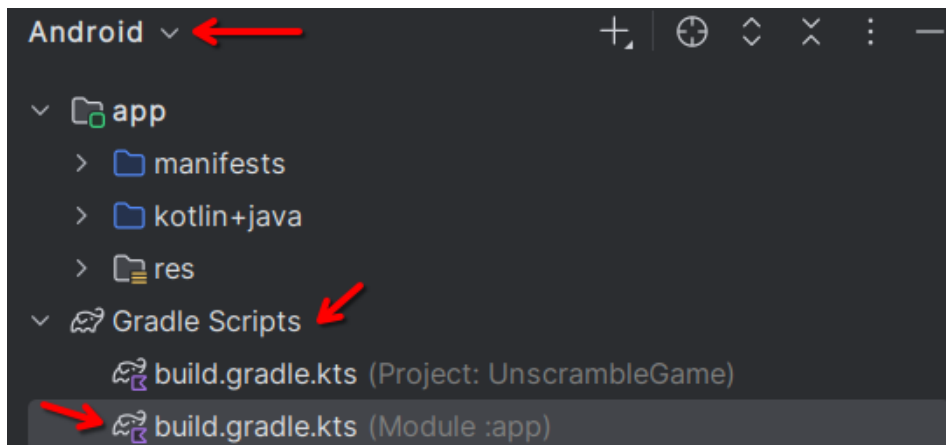
Шаг 1. Добавление зависимостей **ViewModel**

Чтобы использовать **ViewModel** и **StateFlow**, нужно подключить специальные библиотеки.

1.1. Открытие файла **build.gradle**

1. В левой панели Android Studio (режим **Android**)

2. Откройте **Gradle Scripts** → **build.gradle.kts (Module :app)**



3. Найдите блок **dependencies** { ... }

1.2. Добавление зависимостей

Добавьте следующие строки в блок dependencies:

```
implementation("androidx.lifecycle:lifecycle-viewmodel-compose:2.10.0")
```

```
implementation("androidx.lifecycle:lifecycle-runtime-compose:2.10.0")
```

```
dependencies {  
    implementation(libs.androidx.core.ktx)  
    implementation(libs.androidx.lifecycle.runtime.ktx)  
    +implementation("androidx.lifecycle:lifecycle-viewmodel-compose:2.10.0")  
    implementation("androidx.lifecycle:lifecycle-runtime-compose:2.10.0")  
}
```

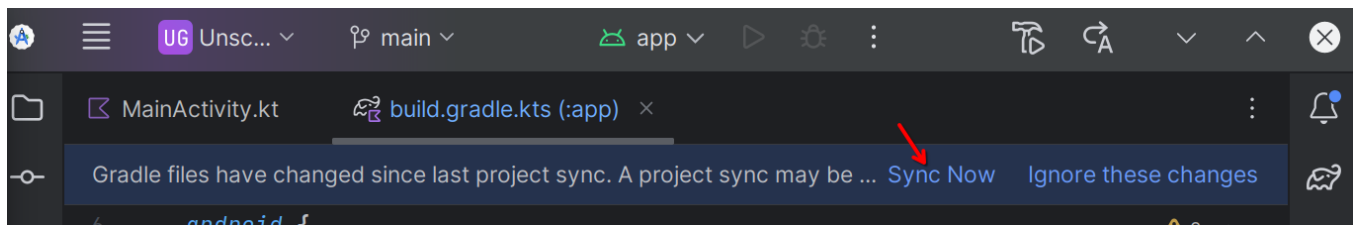
Объяснение:

- **lifecycle-viewmodel-compose** — поддержка **ViewModel** для Jetpack Compose
- **lifecycle-runtime-compose** — интеграция **StateFlow** с **Compose** (автоматическое обновление UI)

1.3. Синхронизация проекта

1. В правом верхнем углу появится панель “Sync Now”

2. Нажмите “Sync Now” и дождитесь завершения



3. Убедитесь, что синхронизация прошла успешно (в нижней панели не должно быть ошибок)

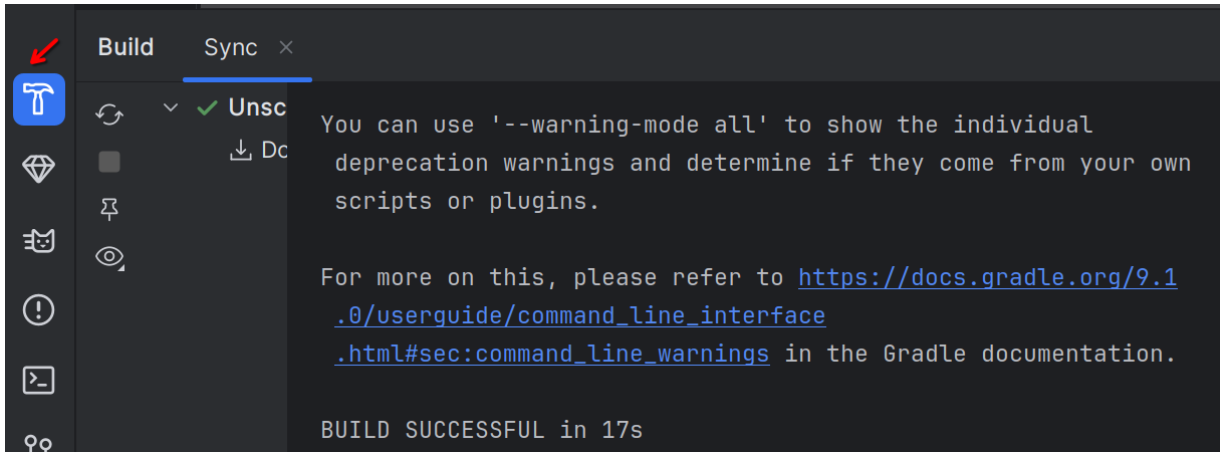
Важно: Если возникли ошибки, проверьте версии библиотек и подключение к интернету.

Скриншоты для отчёта



Сделайте скриншоты:

- Открытый файл **build.gradle.kts** с добавленными зависимостями
- Успешная синхронизация (сообщение в нижней панели)



Сохраните как: **step1_dependencies_FIO.png**

Выполните в терминале:

git add .

git commit -m "Step 1: Add ViewModel dependencies"

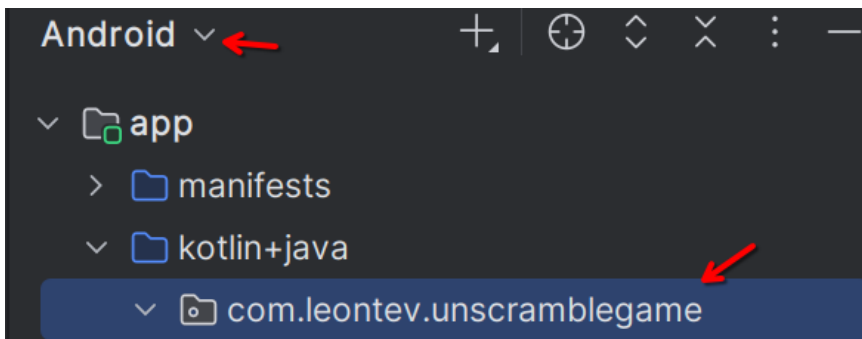
git push

Шаг 2. Создание data class для UI State

Сейчас мы создадим специальную структуру данных, которая будет хранить всё состояние нашей игры.

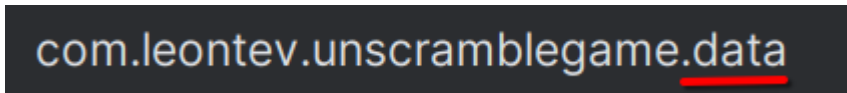
2.1. Создание пакета data

1. В режиме **Android** перейдите в пакет с вашим проектом:

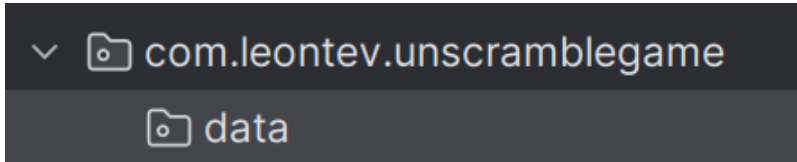


2. Правой кнопкой мыши → **New** → **Package**

3. Введите: **data**



4. Нажмите **ОК**. В итоге у вас появится новый пакет:



Объяснение: Мы создаём отдельный пакет **data** для хранения моделей данных. Это хорошая практика — разделять код по назначению.

2.2. Создание файла **GameUiState**

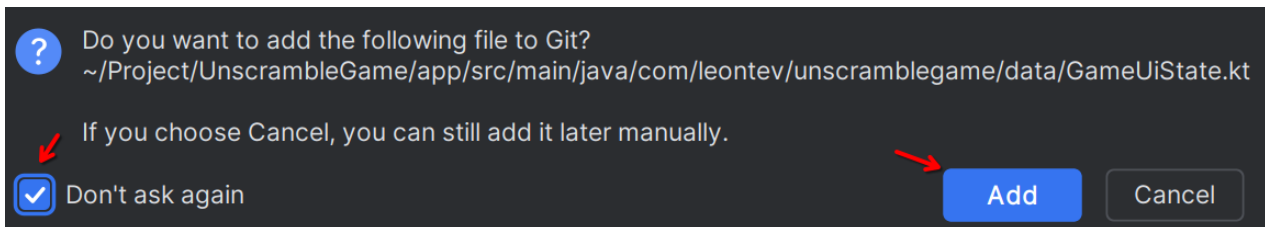
1. Правой кнопкой на пакет **data** → **New** → **Kotlin Class/File**

2. Выберите **File**

3. Имя файла: **GameUiState**

4. Нажмите **Enter**

5. На вопрос добавления файлов в отслеживание Git, можете сразу поставить галочку и нажать **Add**:



2.3. Написание кода **GameUiState**

Что мы делаем: Создаём **data class**, который будет описывать состояние игры. Вместо множества отдельных переменных (`score`, `word`, `isGameOver`), мы объединим их в одну структуру.

Добавьте следующий код (**не трогая первую строчку кода с вашим пакетом**):

```
package com.leontev.unscramblegame.data
```

Ваш Пакет ↗

```
data class GameState(  
    val currentScrambledWord: String = "",  
    val currentWordCount: Int = 1,  
    val score: Int = 0,  
    val isGuessedWordWrong: Boolean = false,  
    val isGameOver: Boolean = false  
)
```

- **data class** — специальный тип класса в Kotlin, который автоматически создаёт методы `equals()`, `hashCode()`, `toString()`, `copy()`. Идеально подходит для хранения данных.
- **currentScrambledWord** — текущее перемешанное слово, которое игрок должен отгадать (например, “nrdaId” вместо “Android”)
- **currentWordCount** — номер текущего вопроса. Начинается с 1. Используется для отображения “Вопрос 1 из 10”
- **score** — количество очков игрока. Увеличивается при правильном ответе
- **isGuessedWordWrong** — флаг, показывающий, что игрок ввёл неправильный ответ. Используется для отображения красной ошибки
- **isGameOver** — флаг окончания игры. Когда `true`, показывается диалог с результатами

Почему используем значения по умолчанию?

- При создании объекта `GameState()` все поля получают начальные значения
- Можно создать объект частично: `GameState(score = 20)` — только `score` изменится, остальное будет по умолчанию

Скриншоты для отчёта



Сделайте скриншот:

- Созданного файла `GameState.kt` с кодом
- Структура пакетов слева (должен быть виден пакет `data`)

Сохраните как: **step2_uistate_FIO.png**

Выполните в терминале:

git add .

git commit -m "Step 2: Create GameState data class"

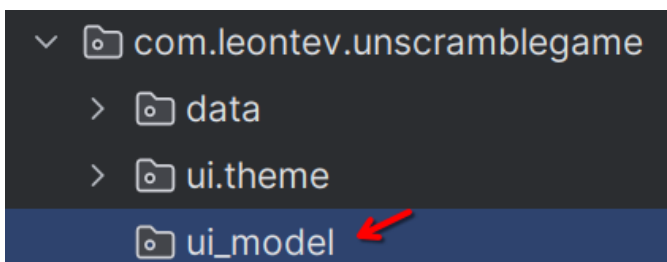
git push

Шаг 3. Создание класса ViewModel

Теперь создадим главный класс нашей игры — ViewModel, который будет управлять логикой и состоянием.

3.1. Создание пакета ui

1. Правой кнопкой на ваш **основной пакет** → **New** → **Package**
2. Введите: **ui_model**
3. Нажмите **OK**



Объяснение: Пакет **ui_model** будет содержать классы, связанные с пользовательским интерфейсом (ViewModel, Composable-функции).

3.2. Создание файла GameViewModel

1. Правой кнопкой на пакет **ui_model** → **New** → **Kotlin Class/File**
2. Выберите **Class**
3. Имя класса: **GameViewModel**
4. Нажмите **Enter**

3.3. Базовая структура ViewModel

Что мы делаем: Создаём класс, который наследуется от ViewModel. Этот класс будет хранить состояние игры и управлять логикой.

Обновим класс GameViewModel:

```

class GameViewModel : ViewModel() {
    private val _uiState = MutableStateFlow(GameUiState())
    val uiState: StateFlow<GameUiState> = _uiState.asStateFlow()

    init {
        resetGame()
    }

    fun resetGame() {
        // TODO: Будем реализовывать дальше
    }
}

```

Необходимые импорты базового класса ViewModel из Android Jetpack и классов для работы с StateFlow:

```

import androidx.lifecycle.ViewModel
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow

```

Также импорт для data-класса с **вашим** пакетом:

```

import com.ваш_пакет.unscramblegame.data.GameUiState

```

Объяснение :

- **class GameViewModel : ViewModel()** — наш класс наследуется от **ViewModel**.
- **private val _uiState** — **приватная** переменная. Это изменяемое состояние (**MutableStateFlow**), которое можем менять только внутри ViewModel
- **val uiState** — **публичная** переменная. Это неизменяемое представление (**StateFlow**), которое UI может только читать. Метод **asStateFlow()** превращает **MutableStateFlow** в **StateFlow**

Важный паттерн “backing property”:

```

private val _uiState = MutableStateFlow(...) // Изменяемое (только для ViewModel)
val uiState: StateFlow<...> = _uiState.asStateFlow() // Только для чтения (для UI)

```

Это защищает от случайного изменения состояния из UI. UI может только **наблюдать** за изменениями, но не может **изменять** состояние напрямую.

- **init { }** — блок инициализации. Выполняется сразу при создании объекта **ViewModel**. Здесь мы вызываем **resetGame()** для начальной настройки игры

Скриншоты для отчёта



Сделайте скриншот:

- Созданного файла GameViewModel.kt с кодом
- Структура пакетов (должны быть видны data и ui)

Сохраните как: **step3_viewmodel_base_FIO.png**

Выполните в терминале:

git add .

git commit -m "Step 3: Create base GameViewModel class"

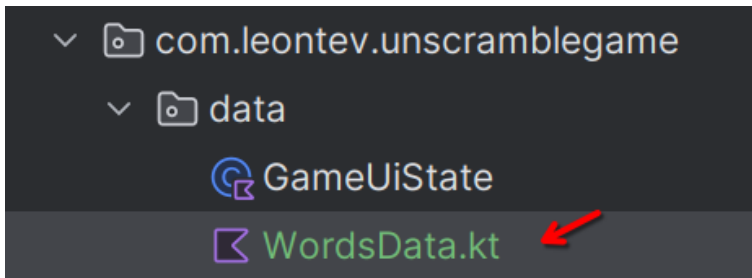
git push

Шаг 4. Добавление данных и логики игры

Теперь добавим список слов для игры и функции для их обработки.

4.1. Создание константного файла

1. Правой кнопкой на пакет **data** → **New** → **Kotlin Class/File**
2. Выберите **File**
3. Имя файла: **WordsData**
4. Нажмите **Enter**



4.2. Список слов

Что мы делаем: Создаём список слов, которые игрок будет угадывать. Можете добавить свои слова!

Добавьте следующий код (**не удаляйте начальный пакет**):

// Максимальное количество слов в игре

const val MAX_NO_OF_WORDS = 10

// Количество очков за правильный ответ

const val SCORE_INCREASE = 20

// Список слов для игры

```
val allWords: List<String> = listOf(
    "android", "kotlin", "compose", "jetpack", "viewmodel", "coroutine", "firebase",
    "material", "database", "network", "retrofit", "room", "navigation", "lifecycle",
    "dependency", "injection", "testing", "debugging", "gradle", "studio"
)
```

Объяснение:

- **const val** — константа времени компиляции. Значение известно до запуска программы и не может измениться
- **val allWords** — неизменяемый список слов (val = cannot reassign, но можно добавлять в список, если бы это был MutableList)
- Слова для игры. Можете добавить свои термины из Android-разработки или любые другие слова.

Примечание: Мы создали более 10 слов (у нас 20), чтобы игра не повторялась. ViewModel будет случайно выбирать 10 слов из этого списка.

4.3. Добавление вспомогательных функций в ViewModel

Откройте файл **GameViewModel.kt** и добавьте следующие поля и функции:

1) Добавьте приватные поля класса (после объявления uiState):

```
class GameViewModel : ViewModel() {
    private val _uiState = MutableStateFlow(GameUiState())
    val uiState: StateFlow<GameUiState> = _uiState.asStateFlow()
    // Текущее слово, которое игрок должен угадать (Неперемешанное)
    +private lateinit var currentWord: String
    // Множество уже использованных слов (чтобы не повторяться)
    +private var usedWords: MutableSet<String> = mutableSetOf()
```

Объяснение:

- **lateinit var currentWord** — переменная, которая будет инициализирована позже (в функции pickRandomWordAndShuffle()). **lateinit** говорит компилятору: “Я обещаю, что инициализирую эту переменную до того, как её использую”. Без неё у вас будет ошибка:



```
private var currentWord: String
```

- **usedWords: MutableSet<String>** — множество для хранения уже использованных слов. Set не допускает дубликатов, поэтому идеально подходит для этой задачи

2) Внутри класса добавьте функцию для перемешивания букв в слове **shuffleCurrentWord()**:

```
private fun shuffleCurrentWord(word: String): String {  
    val tempWord = word.toCharArray() // Преобразуем строку в массив символов  
    tempWord.shuffle() // Перемешиваем массив  
    // Проверяем, что перемешанное слово != исходному  
    while (String(tempWord) == word) {  
        tempWord.shuffle()  
    }  
    return String(tempWord) // Преобразуем массив обратно в строку  
}
```

- **toCharArray()** — превращает String в массив символов. Например: "android" → ['a','n','d','r','o','i','d']

- **shuffle()** — перемешивает элементы массива случайным образом

- Проверяем, что после перемешивания получилось другое слово (а не то же самое по случайности). Если одинаковое — перемешиваем снова

- **String(tempWord)** — преобразуем массив символов обратно в строку

Внутри класса добавьте функцию **pickRandomWordAndShuffle()** для выбора случайного слова из списка и перемешивания его:

```
private fun pickRandomWordAndShuffle(): String {  
    // Выбираем случайное слово, которое ещё не использовали  
    currentWord = allWords.random()  
    // Если слово уже было использовано, выбираем другое  
    while (usedWords.contains(currentWord)) {  
        currentWord = allWords.random()  
    }  
    // Добавляем слово в список использованных  
    usedWords.add(currentWord)  
    // Перемешиваем буквы в слове  
    return shuffleCurrentWord(currentWord)  
}
```

Необходимый импорт:

```
import com.ваш_пакет.unscramblegame.data.allWords
```

- **allWords.random()** — выбирает случайный элемент из списка
- Цикл **while** продолжается, пока мы находим уже использованное слово. Это гарантирует уникальность

- Добавляем слово в Set использованных слов

- return возвращаем перемешанное слово

3) Теперь реализуем ранее созданную функцию **resetGame()**, которая будет сбрасывать игру к начальному состоянию внутри класса:

```
fun resetGame() {  
    + [ usedWords.clear() // Очищаем список использованных слов  
      _uiState.value = GameUiState(  
        currentScrambledWord = pickRandomWordAndShuffle()  
      )  
}
```

- Очищаем Set использованных слов, чтобы можно было играть заново

- Создаём новое состояние **GameUiState** с перемешанным словом. Все остальные поля (score, wordCount и т.д.) получают значения по умолчанию из data class

Скриншоты для отчёта



Сделайте скриншот:

- Файл **WordsData.kt** со списком слов

- Файл **GameViewModel.kt** с добавленными функциями

Сохраните как: **step4_game_logic_FIO.png**

Выполните в терминале:

git add .

git commit -m "Step 4: Add words data and game logic"

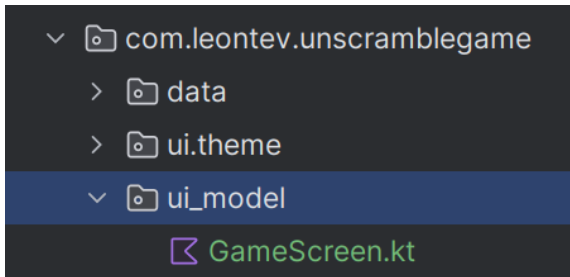
git push

Шаг 5. Подключение ViewModel к UI

Сейчас мы подключим наш **ViewModel** к пользовательскому интерфейсу и создадим экран игры.

5.1. Создание GameScreen Composable

1. Правой кнопкой на пакет `ui_model` → **New** → **Kotlin Class/File**
2. Выберите **File**
3. Имя файла: **GameScreen**
4. Нажмите **Enter**



Напишем следующий код

1. GameScreen — главный экран игры

```
@Composable
fun GameScreen(
    modifier: Modifier = Modifier,
    gameViewModel: GameViewModel = viewModel()
) {}
```

Это **точка входа** экрана. Она собирает данные из `ViewModel` и **передает** их дальше в дочерние **composable**-функции.

Необходимые импорты:

```
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.lifecycle.viewmodel.compose.viewModel
```

Внутри функции **GameScreen()** делаем подписку на состояние **ViewModel**:

```
) {
    + val gameUiState by gameViewModel.uiState.collectAsState()
}
```

- Берём `uiState` из `GameViewModel`
- Подписываемся на него с помощью `collectAsState()`
- Теперь **при любом изменении состояния** экран перерисовуется

Проще: «Следи за состоянием игры и автоматически обновляй UI»

Необходимые импорты:

```
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
```

Создаём основной контейнер экрана — **Column** внутри нашей функции **GameScreen()**:

```
@Composable
fun GameScreen(
    ...
) {
    val gameUiState by gameViewModel.uiState.collectAsState()
    Column(
        modifier = Modifier
            .fillMaxSize()
            .verticalScroll(rememberScrollState())
            .padding(16.dp),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.spacedBy(16.dp)
    ) {
    }
}
```

1. **Column** — элементы будут располагаться **вертикально**
2. **fillMaxSize()** — растягиваемся на весь экран
3. **verticalScroll()** — разрешаем прокрутку
4. **padding(16.dp)** — отступы от краёв экрана
5. **spacedBy(16.dp)** — расстояние между элементами

Необходимые импорты:

```
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.verticalScroll
import androidx.compose.ui.unit.dp
import androidx.compose.ui.Alignment
```

Внутри **Column** создадим заголовок игры:

```

@Composable
fun GameScreen(
    ...
) {
    val gameUiState by gameViewModel.uiState.collectAsState()
    Column(...) {
        Text(
            text = "Unscramble Game",
            style = MaterialTheme.typography.titleLarge,
            fontSize = 32.sp
        )
    }
}

```

- отображает название игры
- использует крупный шрифт
- центрируется благодаря Column

Необходимые импорты:

```

import androidx.compose.material3.Text
import androidx.compose.material3.MaterialTheme
import androidx.compose.ui.unit.sp

```

3. Создадим **функции заглушки** для статуса игры и игрового макета:

```

@Composable
fun GameScreen(...) {...}

@Composable
fun GameStatus() {}

@Composable
fun GameLayout(){}

```

4. GameStatus — статус игры (счёт и номер слова)

Добавим в функцию **GameStatus()** свойства:

```

@Composable
fun GameStatus(
    wordCount: Int,
    score: Int,
    modifier: Modifier = Modifier
) {}

```

Внутри тела функции добавим показ **информационной карточки**:

```
@Composable
fun GameStatus(
    ...
) {
    Card(
        modifier = modifier.fillMaxWidth(),
        colors = CardDefaults.cardColors(
            containerColor = MaterialTheme.colorScheme.primaryContainer
        )
    ) { }
}
```

- на всю ширину экрана
- с цветом из темы приложения

Необходимые импорты:

```
import androidx.compose.material3.Card
import androidx.compose.material3.CardDefaults
import androidx.compose.foundation.layout.fillMaxWidth
```

Внутри карточки добавляем **Row** для выравнивания элементов **по горизонтали**:

```
Card(...)
) {
    Row(
        modifier = Modifier
            .fillMaxWidth()
            .padding(16.dp),
        horizontalArrangement = Arrangement.SpaceBetween
    ) { }
}
```

- элементы будут **по горизонтали**
- **SpaceBetween** — один слева, другой справа

Необходимые импорты:

```
import androidx.compose.foundation.layout.Row
```

Внутри **Row** добавим **два** текста:

```
Row(...) {
    Text(
        text = "Слово $wordCount из ${com.leontev.unscramblegame.data
            .MAX_NO_OF_WORDS}",
        style = MaterialTheme.typography.titleMedium
    )
    Text(
        text = "Счёт: $score",
        style = MaterialTheme.typography.titleMedium
    )
}
```

- **Первый текст** отображает номера текущего слова и общее количество слов
- **Второй текст** счёта показывает текущие очки игрока

Итоговый вид функции **GameStatus()**:

```
@Composable
fun GameStatus(
    wordCount: Int,
    score: Int,
    modifier: Modifier = Modifier) {
    Card(...) {
        Row(...) {
            Text(...)
            Text(...)
        }
    }
}
```

5. Перейдём к написанию функции **GameLayout()** — основной игровой интерфейс

Она будет отвечать за:

- отображение перемешанного слова
- ввод ответа
- кнопки управления

Добавляем **свойства**:

```
@Composable
fun GameLayout(
    currentScrambledWord: String,
    onUserGuessChanged: (String) -> Unit,
    onKeyboardDone: () -> Unit,
    modifier: Modifier = Modifier
) {
}
```

Добавляем в **тело функции** локальное состояние ввода пользователя:

```
@Composable
fun GameLayout(
    ...
) {
    + var userGuess by remember { mutableStateOf("") }
}
```

- создаётся **локальное состояние**
- **remember** сохраняет значение между перерисовками
- **userGuess** хранит текст, введённый пользователем

Необходимые импорты:

```
import androidx.compose.runtime.remember
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.setValue
import androidx.compose.runtime.getValue
```

Напишем основную колонку макета:

```
) {
    var userGuess by remember { mutableStateOf("") }
    + Column(
        modifier = modifier.fillMaxWidth(),
        verticalArrangement = Arrangement.spacedBy(16.dp),
        horizontalAlignment = Alignment.CenterHorizontally
    ) { }
}
```

Элементы:

- выстроены вертикально
- с равными отступами
- выровнены по центру

Внутри **Column** добавляем карточку с перемешанным словом:

```
Column(...) {  
    Card(  
        modifier = Modifier  
            .fillMaxWidth()  
            .height(150.dp),  
        colors = CardDefaults.cardColors(  
            containerColor = MaterialTheme.colorScheme.surfaceVariant  
        )  
    ) { }  
}
```

- карточка фиксированной высоты
- на всю ширину экрана

Необходимые импорты:

```
import androidx.compose.foundation.layout.height
```

Внутри карточки добавляем для центрирования слова – **Box**:

```
{  
    Card(...) {  
        Box(  
            modifier = Modifier.fillMaxSize(),  
            contentAlignment = Alignment.Center  
        ) { }  
    }  
}
```

- **Box** нужен, чтобы растянуться на всю карточку и **выровнять текст по центру**

Необходимые импорты:

```
import androidx.compose.foundation.layout.Box
```

Внутри **Box** добавим отображение перемешанного слова:

```
Card(...) {  
    Box(...) {  
        Text(  
            text = currentScrambledWord,  
            style = MaterialTheme.typography.displayMedium,  
            fontSize = 45.sp  
        )  
    }  
}
```

Теперь добавьте за **пределами** карточки (внимательно посмотрите куда вставлять в коде) **текст**, который показывает текущее перемешанное слово крупным шрифтом.:

```
Column(...
) {
    Card(...) {
        Box(...) {
            Text(...)
        }
    }
    Text(
        text = "Разгадайте слово",
        style = MaterialTheme.typography.titleMedium
    )
}
```

Далее под ним добавим поле ввода (**OutlinedTextField**):

```
Column(...) {
    Card(...) {...}
    Text(...)
    OutlinedTextField(
        value = userGuess,
        onValueChange = {
            userGuess = it
            onUserGuessChanged(it)
        },
        singleLine = true,
        modifier = Modifier.fillMaxWidth(),
        label = { Text("Введите слово") },
        keyboardOptions = KeyboardOptions.Default.copy(
            imeAction = ImeAction.Done
        ),
        keyboardActions = KeyboardActions(
            onDone = { onKeyboardDone() }
        )
    )
}
```

Необходимые импорты:

```
import androidx.compose.material3.OutlinedTextField
import androidx.compose.foundation.text.KeyboardOptions
import androidx.compose.ui.text.input.ImeAction
import androidx.compose.foundation.text.KeyboardActions
```


- При вводе текста:
 1. обновляется локальное состояние **userGuess**
 2. вызывается колбэк, чтобы сообщить наружу
- Клавиатура и кнопка **Done**

Когда пользователь нажимает **Done**:

- вызывается **onKeyboardDone()**

Далее добавим кнопку «Проверить»

```
Column(...) {
    Card(...) {...}
    Text(...)
    OutlinedTextField(...)
    Button(
        onClick = { /* TODO */ },
        modifier = Modifier.fillMaxWidth()
    ) {
        Text(
            text = "Проверить",
            fontSize = 16.sp
        )
    }
}
```

- Будет использоваться для проверки ответа пользователя.

Необходимые импорты:

```
import androidx.compose.material3.Button
```

Добавляем кнопку «Пропустить»

```
Column(...) {
    Card(...) {...}
    Text(...)
    OutlinedTextField(...)
    Button(...) {...}
    OutlinedButton(
        onClick = { /* TODO */ },
        modifier = Modifier.fillMaxWidth()
    ) {
        Text(
            text = "Пропустить",
            fontSize = 16.sp
        )
    }
}
```

- Позволит перейти к следующему слову.

Необходимые импорты:

```
import androidx.compose.material3.OutlinedButton
```

Итоговый вид функции **GameLayout()**:

```
@Composable
fun GameLayout(...) {
    var userGuess by remember { mutableStateOf("") }
    Column(...) {
        Card(...) {...}
        Text(...)
        OutlinedTextField(...)
        Button(...) {...}
        OutlinedButton(...) {...}
    }
}
```

Возвращаемся к функции **GameScreen()**. Вставим блок статуса игры:

```
@Composable
fun GameScreen(...) {
    val gameUiState by gameViewModel.uiState.collectAsState()
    Column(...) {
        Text(...)
        + GameStatus(
            wordCount = gameUiState.currentWordCount,
            score = gameUiState.score
        )
    }
}
```

- вставляем другой **composable**
- передаём в него:
 - номер текущего слова
 - текущий счёт

GameScreen не рисует статус сам, а делегирует это **GameStatus**

Далее вставим основной игровой макет:

```

@Composable
fun GameScreen(...) {
    val gameUiState by gameViewModel.uiState.collectAsState()
    Column(...) {
        Text(...)
        GameStatus(...)
        GameLayout(
            currentScrambledWord = gameUiState.currentScrambledWord,
            onUserGuessChanged = { }, // TODO: добавим позже
            onKeyboardDone = { } // TODO: добавим позже
        )
    }
}

```

- передаётся **перемешанное слово**
- передаются колбэки (пока пустые)
- GameLayout отвечает за:
 - отображение слова
 - поле ввода
 - кнопки

5.2. Обновление MainActivity

1. Откройте **MainActivity.kt** и удалите лишние функции, оставив базовую структуру:

```

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            UnscrambleGameTheme {
                // ...
            }
        }
    }
}

```

2. Нажмите **Ctrl+Alt+O**, чтобы Android Studio автоматически удалила неиспользуемые импорты.

3. Замените функцию **setContent()**:

```

setContent {
    UnscrambleGameTheme {
        Scaffold(
            modifier = Modifier.fillMaxSize()
        ) { innerPadding ->
            GameScreen(
                modifier = Modifier.padding(innerPadding)
            )
        }
    }
}

```

4. Добавьте по необходимости **импорты**

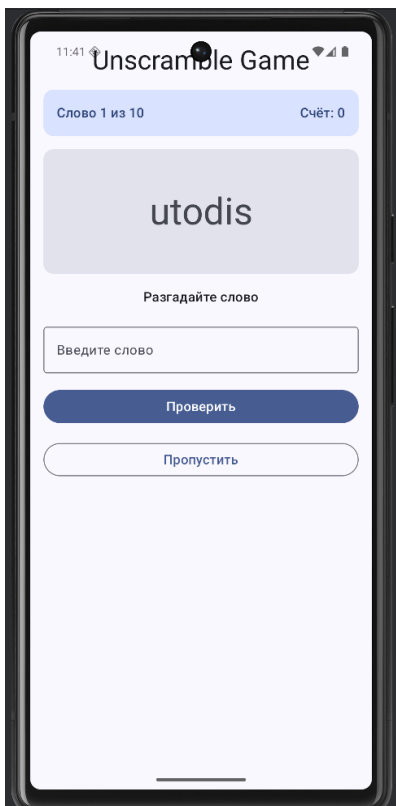
Объяснение:

- Заменяли стандартный Greeting на наш GameScreen
- Scaffold — стандартный контейнер Material Design, обеспечивает правильные

отступы

5.3. Запуск приложения

1. Нажмите **Run** или **Shift + F10**
2. Выберите эмулятор или подключённое устройство
3. Дождитесь запуска приложения



Что должно работать:

- Отображается перемешанное слово
- Можно вводить текст в поле
- Счётчик вопросов показывает “Слово 1 из 10”
- Счёт равен 0

Проверка сохранения состояния:

1. Поверните экран эмулятора (кнопка rotate в панели)
2. **Важно:** Перемешанное слово **НЕ должно измениться!** Это работа ViewModel!

Скриншоты для отчёта



Сделайте скриншоты:

1. Запущенное приложение в портретной ориентации
2. То же приложение после поворота экрана в **landscape**
3. Убедитесь, что слово **не изменилось**
4. Сохраните как: **step5_ui_portrait_FIO.png** и **step5_ui_landscape_FIO.png**

Выполните в терминале:

git add .

git commit -m "Step 5: Connect ViewModel to UI"

git push

Шаг 6. Реализация проверки ответа

Теперь добавим логику проверки правильности ответа игрока.

6.1. Добавление функций в ViewModel

Откройте **GameViewModel.kt** и добавьте следующие функции:

- 1) Добавьте переменную для хранения текущего ответа пользователя:

```
class GameViewModel : ViewModel() {
    private val _uiState = MutableStateFlow(GameUiState())
    val uiState: StateFlow<GameUiState> = _uiState.asStateFlow()
    private lateinit var currentWord: String
    private var usedWords: MutableSet<String> = mutableSetOf()

    + [ var userGuess by mutableStateOf("")
      private set
```

- **var userGuess** — текущий текст, который вводит пользователь
- **private set** — можем менять значение только внутри класса, но читать могут все

Необходимые импорты:

```
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.getValue
import androidx.compose.runtime.setValue
```

2) В класс GameViewModel добавьте функцию обновления ответа пользователя:

```
fun updateUserGuess(guessedWord: String) {
    userGuess = guessedWord
}
```

3) В класс GameViewModel добавьте функцию обновления состояния игры:

```
private fun updateGameState(updatedScore: Int) {
    if (usedWords.size == MAX_NO_OF_WORDS) {
        // Достигли максимального количества слов — игра окончена
        _uiState.update { currentState ->
            currentState.copy(
                isGuessedWordWrong = false,
                score = updatedScore,
                isGameOver = true
            )
        }
    } else {
        // Переходим к следующему слову
        _uiState.update { currentState ->
            currentState.copy(
                isGuessedWordWrong = false,
                currentScrambledWord = pickRandomWordAndShuffle(),
                score = updatedScore,
                currentWordCount = currentState.currentWordCount + 1
            )
        }
    }
}
```

- проверяем, использовали ли мы уже 10 слов
- Если да — обновляем состояние:
 - `isGuessedWordWrong = false` — убираем ошибку (если была)
 - `score = updatedScore` — обновляем счёт
 - `isGameOver = true` — помечаем, что игра окончена
- Если нет — переходим к следующему слову:
 - Берём новое перемешанное слово
 - Увеличиваем счётчик вопросов на 1
 - Обновляем счёт

4) В класс `GameViewModel` добавьте функцию проверки ответа:

```
fun checkUserGuess() {
    if (userGuess.equals(currentWord, ignoreCase = true)) {
        // Правильный ответ!
        val updatedScore = _uiState.value.score + SCORE_INCREASE
        updateGameState(updatedScore)
    } else {
        // Неправильный ответ
        _uiState.update { currentState ->
            currentState.copy(isGuessedWordWrong = true)
        }
    }

    // Очищаем поле ввода
    updateUserGuess("")
}
```

- `equals(..., ignoreCase = true)` — сравнивает строки без учёта регистра. “Android” == “android” == “ANDROID”
- Вычисляем новый счёт (текущий + `SCORE_INCREASE`)
- Вызываем функцию обновления состояния (создадим следующей)
- `_uiState.update { }` — специальная функция для обновления `StateFlow`. Она получает **текущее состояние**, а мы возвращаем **новое состояние**

- `copy(isGuessedWordWrong = true)` — создаёт копию `GameUiState` с изменённым одним полем. Это функция `data class`. Все остальные поля останутся прежними
- Очищаем поле ввода после проверки

5) Добавьте функцию пропуска слова:

```
fun skipWord() {  
    updateGameState(_uiState.value.score)  
    // Сбрасываем текст пользователя  
    updateUserGuess("")  
}
```

- Вызываем `updateGameState` с **текущим** счётом (не увеличиваем)
- Очищаем поле ввода

По итогу класс **GameViewModel** будет выглядеть следующим образом:

```
class GameViewModel : ViewModel() {  
    private val _uiState = MutableStateFlow(GameUiState())  
    val uiState: StateFlow<GameUiState> = _uiState.asStateFlow()  
    private lateinit var currentWord: String  
    private var usedWords: MutableSet<String> = mutableSetOf()  
    var userGuess by mutableStateOf("")  
        private set  
  
    fun updateUserGuess(guessedWord: String) {...}  
    fun checkUserGuess() {...}  
    private fun updateGameState(updatedScore: Int) {...}  
    fun skipWord() {...}  
    private fun pickRandomWordAndShuffle(): String {...}  
    private fun shuffleCurrentWord(word: String): String {...}  
    init {...}  
    fun resetGame() {...}  
}
```

6.2. Подключение функций к UI

Откройте **GameScreen.kt**, обновим внутри код.

1) Обновите функцию **GameLayout**.

Добавим параметры:


```

@Composable
fun GameLayout(
    currentScrambledWord: String,
    +userGuess: String, // +
    onUserGuessChanged: (String) -> Unit,
    onKeyboardDone: () -> Unit,
    +isGuessWrong: Boolean, // +
    +onSubmitClicked: () -> Unit, // +
    +onSkipClicked: () -> Unit, // +
    modifier: Modifier = Modifier
) {

```

Внесём изменения в **OutlinedTextField**:

```

OutlinedTextField(
    value = userGuess,
    onValueChange = {
        userGuess = it
        onUserGuessChanged(it)
    },
    singleLine = true,
    modifier = Modifier.fillMaxWidth(),
    label = { Text("Введите слово") },
    +isError = isGuessWrong, // добавили

```

Добавим сообщение об ошибке после **OutlinedTextField** и перед **Button**:

```

    )
    )
    // Добавляем сообщение об ошибке
    if (isGuessWrong) {
        Text(
            text = "Неправильно! Попробуйте ещё раз.",
            color = MaterialTheme.colorScheme.error,
            style = MaterialTheme.typography.bodyMedium
        )
    }
    Button(

```

Вносим изменения в **Button**:

```
Button(  
    • onClick = onSubmitClicked, // изменили  
    modifier = Modifier.fillMaxWidth()  
) {
```

Изменения в **OutlinedButton**:

```
OutlinedButton(  
    onClick = onSkipClicked, // изменили
```

2) Обновите внутри **GameScreen** функцию **GameLayout()**:

```
@Composable  
fun GameScreen(...  
) {  
    val gameUiState by gameViewModel.uiState.collectAsState()  
    Column(...) {  
        Text(...)  
        GameStatus(...)  
        GameLayout(  
            currentScrambledWord = gameUiState.currentScrambledWord,  
            userGuess = gameViewModel.userGuess, // добавили  
            ✓ onUserGuessChanged = { gameViewModel.updateUserGuess(it) }, // добавили  
            onKeyboardDone = { gameViewModel.checkUserGuess() }, // добавили  
            isGuessWrong = gameUiState.isGuessedWordWrong, // добавили  
            onSubmitClicked = { gameViewModel.checkUserGuess() }, // добавили  
            onSkipClicked = { gameViewModel.skipWord() } // добавили  
        )  
    }  
}
```

- **userGuess** теперь приходит из **ViewModel**, а не хранится локально
- **isError = isGuessWrong** — **TextField** становится красным при ошибке
- **Строки 58-64**: Показываем сообщение об ошибке, если ответ неправильный

6.3. Тестирование

1. Запустите приложение
2. Попробуйте отгадать слово правильно → счёт должен увеличиться на 20
3. Попробуйте ввести неправильное слово → должно появиться красное сообщение

4. Нажмите “Пропустить” → должно появиться новое слово без изменения счёта

5. **Поверните экран** → всё должно остаться на своих местах!



Скриншоты для отчёта



Сделайте скриншоты:

1. Правильный ответ (счёт увеличился, новое слово)
2. Неправильный ответ (красная ошибка)
3. После пропуска слова
4. Сохраните как: **step6_game_working_FIO.png**

Выполните в терминале:

git add .

git commit -m "Step 6: Implement answer checking logic"

git push

Шаг 7. Добавление диалога завершения игры

Когда игрок ответит на все 10 вопросов, нужно показать диалог с результатами и предложением сыграть снова.

7.1. Создание AlertDialog

Откройте **GameScreen.kt** и добавьте новую Composable-функцию:

```
@Composable
fun FinalScoreDialog(
    score: Int,
    onPlayAgain: () -> Unit,
    modifier: Modifier = Modifier
) {
    AlertDialog(
        onDismissRequest = {
            // Пустой блок – диалог можно закрыть только кнопкой
        },
        //Заголовок диалога
        title = { Text(text = "Поздравляем!") },
        // Содержимое – показываем счёт игрока
        text = {
            Column {
                Text(text = "Вы набрали:")
                Text(
                    text = "$score очков",
                    style = MaterialTheme.typography.displaySmall,
                    fontSize = 36.sp
                )
            }
        },
        modifier = modifier,
        dismissButton = {},
        // Кнопка подтверждения – запускает новую игру
        confirmButton = {
            TextButton(onClick = onPlayAgain) {
                Text(text = "Играть снова")
            }
        }
    )
}
```

Необходимые импорты:

```
import androidx.compose.material3.AlertDialog
import androidx.compose.material3.TextButton
```

7.2. Показ диалога в GameScreen

Обновите функцию **GameScreen**:

```
@Composable
fun GameScreen(...) {
    val gameUiState by gameViewModel.uiState.collectAsState()
    Column(...) {
        Text(...)
        GameStatus(...)
        GameLayout(...)
        // Показываем диалог, если игра окончена
        if (gameUiState.isGameOver) {
            FinalScoreDialog(
                score = gameUiState.score,
                onPlayAgain = { gameViewModel.resetGame() }
            )
        }
    }
}
```

- Проверяем флаг `isGameOver`
- Если `true`, показываем диалог с итоговым счётом
- При нажатии “Играть снова” вызываем `resetGame()`, который очистит

использованные слова и создаст новое состояние

7.3. Тестирование полного цикла игры

1. Запустите приложение
2. **Быстрый тест:** Нажмите кнопку “Пропустить” 10 раз подряд
3. Должен появиться диалог с вашим счётом (0 очков, если всё пропустили)
4. Нажмите “Играть снова” → игра должна начаться заново
5. **Полный тест:** Попробуйте набрать очки, ответив на несколько вопросов

правильно

6. **Проверка ViewModel:** Поверните экран в середине игры → состояние должно сохраниться!

Скриншоты для отчёта



Сделайте скриншоты:

1. Диалог завершения игры с вашим счётом
2. Игра после нажатия “Играть снова” (должно быть первое слово, счёт 0)

3. Сохраните как: **step7_game_complete_FIO.png**

Выполните в терминале:

git add .

git commit -m "Step 7: Add game completion dialog"

git push

Шаг 8. Добавление unit-тестов

Хорошая стратегия тестирования строится вокруг покрытия различных путей выполнения и граничных условий вашего кода. На самом базовом уровне тесты можно разделить на три сценария: **путь успеха**, **путь ошибки** и **граничный случай**.

Путь успеха (Success path):

Тесты пути успеха — также известные как *happy path* — фокусируются на проверке функциональности при положительном сценарии выполнения. Положительный сценарий — это сценарий без исключений и ошибок. По сравнению с путём ошибки и граничными случаями, для пути успеха проще составить исчерпывающий список сценариев, поскольку они сосредоточены на ожидаемом поведении приложения.

Пример пути успеха в приложении **Unscramble** — корректное обновление счёта, количества слов и зашифрованного слова, когда пользователь вводит правильное слово и нажимает кнопку **Submit**.

Путь ошибки (Error path):

Тесты пути ошибки направлены на проверку функциональности при отрицательном сценарии — то есть на то, как приложение реагирует на ошибки или некорректный ввод пользователя. Определить все возможные ошибочные сценарии довольно сложно, поскольку при нарушении ожидаемого поведения может возникнуть множество вариантов развития событий.

Один из общих советов — перечислить все возможные пути ошибки, написать для них тесты и продолжать развивать модульные тесты по мере обнаружения новых сценариев. Пример пути ошибки в приложении **Unscramble** — пользователь вводит неправильное слово и нажимает кнопку **Submit**, в результате чего появляется сообщение об ошибке, а счёт и количество слов не обновляются.

Граничный случай (Boundary case):

Граничный случай фокусируется на тестировании предельных условий в приложении. В **Unscramble** примером граничного случая является проверка состояния UI при запуске приложения, а также состояния UI после того, как пользователь сыграл максимальное количество слов.

Создание тестовых сценариев на основе этих категорий может служить ориентиром для вашего плана тестирования.

Создание тестов

Хороший модульный тест обычно обладает следующими четырьмя свойствами:

- **Сфокусированность (Focused):** Тест должен быть сосредоточен на проверке одной единицы — например, фрагмента кода. Чаще всего это класс или метод. Тест должен быть узким и проверять корректность отдельных частей кода, а не нескольких сразу.
- **Понятность (Understandable):** Код теста должен быть простым и понятным при чтении. С первого взгляда разработчик должен понимать цель теста.
- **Детерминированность (Deterministic):** Тест должен стабильно проходить или падать. При любом количестве запусков без изменений в коде результат должен быть одинаковым. Тест не должен быть «нестабильным», когда он то проходит, то падает без изменения кода.
- **Самодостаточность (Self-contained):** Тест не должен требовать участия человека или дополнительной настройки и должен выполняться изолированно.

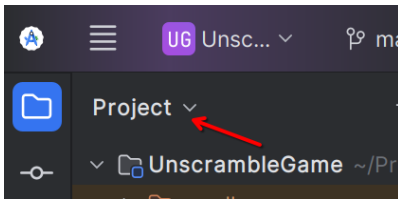
Путь успеха

Чтобы написать модульный тест для пути успеха, необходимо проверить, что при условии, что экземпляр **GameViewModel** был инициализирован, при вызове метода **updateUserGuess()** с правильным словом, за которым следует вызов метода **checkUserGuess()**, выполняется следующее:

- Правильный вариант слова передаётся в метод **updateUserGuess()**.
- Метод **checkUserGuess()** вызывается.
- Значения счёта и статуса **isGuessedWordWrong** обновляются корректно.

8.1. Создание тестового файла

1. Переключитесь в режим **Project**

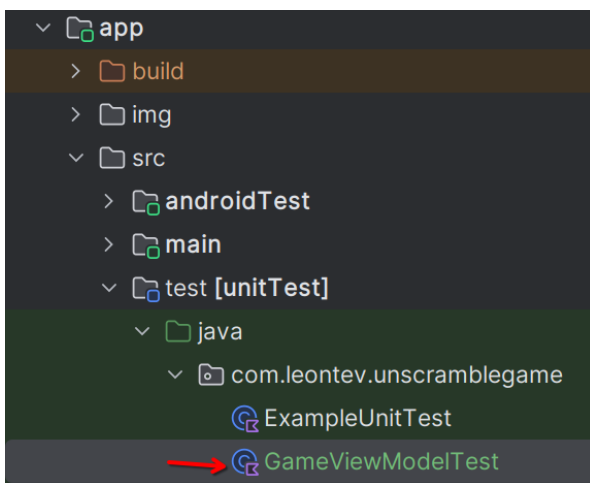


2. Найдите папку **app/src/test[unitTest]/java/com.ваш_пакет**

3. Правой кнопкой → **New** → **Kotlin Class/File**

4. Выберите **Class**

5. Имя: **GameViewModelTest**



8.2. Настройка тестовых зависимостей

1. Откройте **build.gradle.kts (Module :app)** и добавьте в **dependencies**:

```
testImplementation("junit:junit:4.13.2")
```

```
testImplementation("org.jetbrains.kotlinx:kotlinx-coroutines-test:1.7.3")
```

2. Нажмите **Sync Now**.

8.3. Написание базового теста

1. Добавьте необходимые импорты в начале файла **GameViewModelTest.kt**

Обратите внимание на пакеты, которые вам нужно будет заменить на свои:

```
package com.leontev.unscramblegame // тут ваш пакет
// обратите внимание, тут должен быть ваш пакет
import com.leontev.unscramblegame.data.MAX_NO_OF_WORDS
import com.leontev.unscramblegame.data.SCORE_INCREASE
import com.leontev.unscramblegame.ui_model.GameViewModel
import org.junit.Assert.*
import org.junit.Test
```


2. В теле класса **GameViewModelTest** объявите свойство **viewModel** и присвойте ему экземпляр класса **GameViewModel**:

```
class GameViewModelTest {  
    // Создаём новый экземпляр ViewModel для каждого теста  
    private val viewModel = GameViewModel()  
}
```

Чтобы получить правильное (расшифрованное) слово, создайте внутри класса **GameViewModelTest** функцию **getUnscrambledWord()**:

```
private fun getUnscrambledWord(scrambledWord: String): String {  
    // обратите внимание, тут должен быть ваш пакет  
    return com.leontev.unscramblegame.data.allWords.firstOrNull { word ->  
        scrambledWord.toSet() == word.toSet()  
    } ?: ""  
}
```

Функция **getUnscrambledWord()** принимает в качестве аргумента **gameUiState.currentScrambledWord** и возвращает исходное слово. Присвойте возвращаемое значение новой неизменяемой переменной с именем **unScrambledWord**

3. Чтобы написать модульный тест для пути успеха, создайте функцию **gameViewModel_CorrectWordGuessed_ScoreUpdatedAndErrorFlagUnset()** внутри класса **GameViewModelTest** и аннотируйте её аннотацией **@Test**:

```
class GameViewModelTest {  
    private val viewModel = GameViewModel()  
  
    @Test  
    fun gameViewModel_CorrectWordGuessed_ScoreUpdatedAndErrorFlagUnset() {...}
```

Чтобы передать правильное слово, введённое игроком, в метод **viewModel.updateUserGuess()**, необходимо получить правильное расшифрованное слово из зашифрованного слова, хранящегося в **GameUiState**. Для этого сначала получите текущее состояние игрового UI.

- В теле функции создайте переменную **currentGameUiState** и присвойте ей значение **viewModel.uiState.value**:

```
@Test
fun gameViewModel_CorrectWordGuessed_ScoreUpdatedAndErrorFlagUnset() {
    f var currentGameState = viewModel.uiState.value
}
```

Весь следующий код пишем внутри функции

gameViewModel_CorrectWordGuessed_ScoreUpdatedAndErrorFlagUnset()

Чтобы получить правильный вариант слова, введенного игроком, используйте функцию **getUnscrambledWord()**, которая принимает в качестве аргумента **currentGameState.currentScrambledWord** и возвращает расшифрованное слово. Сохраните возвращаемое значение в новой неизменяемой переменной с именем **correctPlayerWord** и присвойте ей результат работы функции **getUnscrambledWord()**:

```
val correctPlayerWord = getUnscrambledWord(currentGameState.currentScrambledWord)
```

Чтобы проверить, что введенное слово является правильным, добавьте вызов метода **viewModel.updateUserGuess()** и передайте в него переменную **correctPlayerWord** в качестве аргумента. Затем добавьте вызов метода **viewModel.checkUserGuess()** для проверки введенного слова:

```
viewModel.updateUserGuess(correctPlayerWord)
viewModel.checkUserGuess()
```

Теперь вы готовы проверить, соответствует ли состояние игры ожидаемому. Получите экземпляр класса **GameState** из значения свойства **viewModel.uiState** и сохраните его в переменной **currentGameState**:

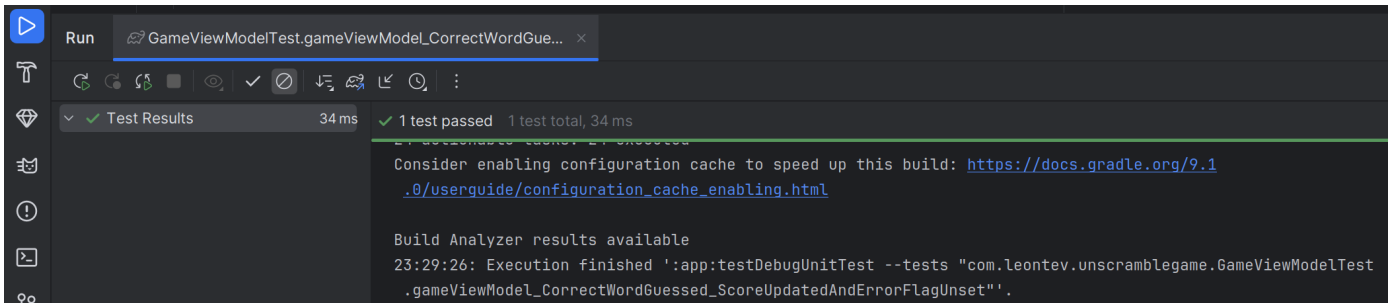
```
currentGameState = viewModel.uiState.value
```

Чтобы проверить, что введенное слово угадано правильно и счёт обновился, используйте функцию **assertFalse()** для проверки, что свойство **currentGameState.isGuessedWordWrong** имеет значение **false**, и функцию **assertEquals()** для проверки, что значение свойства **currentGameState.score** равно **20**:

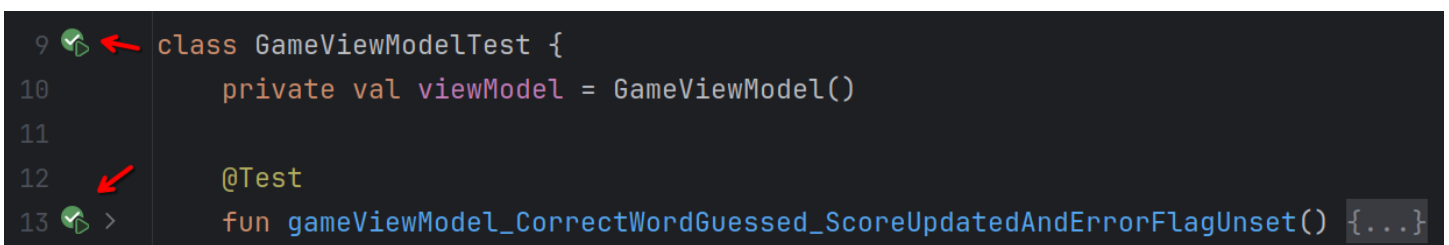
```
assertEquals(SCORE_INCREASE, currentGameState.score)
assertFalse(currentGameState.isGuessedWordWrong)
```

Запустите тест.

Тест должен успешно выполниться, поскольку все утверждения (assertions) корректны:



Успешное выполнение:



Путь ошибки (Error path)

Чтобы написать модульный тест для пути ошибки, необходимо проверить, что при передаче неправильного слова в качестве аргумента в метод `viewModel.updateUserGuess()` и последующем вызове метода `viewModel.checkUserGuess()` происходит следующее:

- Значение свойства `currentGameUiState.score` остаётся неизменным.
- Значение свойства `currentGameUiState.isGuessedWordWrong` устанавливается в `true`, поскольку слово угадано неверно.

Выполните следующие шаги для создания теста:

В теле класса `GameViewModelTest` создайте функцию `gameViewModel_IncorrectGuess_ErrorFlagSet()` и аннотируйте её аннотацией `@Test`:



Определите переменную `incorrectPlayerWord` и присвойте ей значение `"incorrect"`, которого не должно быть в списке слов:

```
@Test
fun gameViewModel_IncorrectGuess_ErrorFlagSet() {
    +val incorrectPlayerWord = "incorrect"
}
```

Весь следующий код пишем внутри функции **gameViewModel_IncorrectGuess_ErrorFlagSet()**

Добавьте вызов метода **viewModel.updateUserGuess()** и передайте в него переменную **incorrectPlayerWord** в качестве аргумента:

```
viewModel.updateUserGuess(incorrectPlayerWord)
```

Добавьте вызов метода **viewModel.checkUserGuess()** для проверки введённого слова:

```
viewModel.checkUserGuess()
```

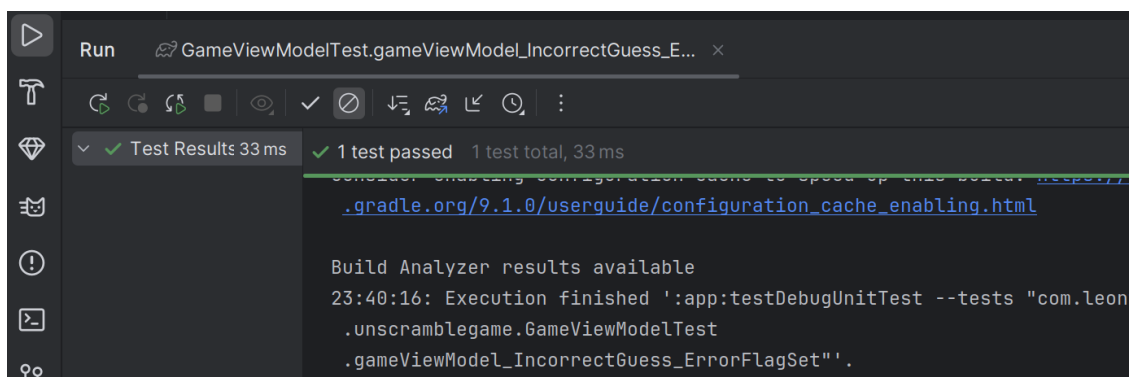
Добавьте переменную **currentGameState** и присвойте ей значение состояния **viewModel.uiState.value**:

```
val currentGameState = viewModel.uiState.value
```

Используйте функции утверждений (**assertions**), чтобы проверить, что значение свойства **currentGameState.score** равно 0, а значение свойства **currentGameState.isGuessedWordWrong** установлено в **true**:

```
assertEquals(0, currentGameState.score)
assertTrue(currentGameState.isGuessedWordWrong)
```

Запустите тест и убедитесь, что он успешно проходит:



```
@Test
fun gameViewModel_IncorrectGuess_ErrorFlagSet() {...}
```

Граничный случай (Boundary case)

Чтобы протестировать начальное состояние пользовательского интерфейса, необходимо написать модульный тест для класса `GameViewModel`. Тест должен подтвердить, что при инициализации `GameViewModel` выполняются следующие условия:

- Свойство `currentWordCount` установлено в 1.
- Свойство `score` установлено в 0.
- Свойство `isGuessedWordWrong` установлено в `false`.
- Свойство `isGameOver` установлено в `false`.

Выполните следующие шаги, чтобы добавить тест:

Создайте метод

`gameViewModel_Initialization_FirstWordLoaded()` и аннотируйте его аннотацией `@Test`:

```
@Test
fun gameViewModel_Initialization_FirstWordLoaded() {...}
```

2. Получите доступ к свойству `viewModel.uiState.value`, чтобы получить начальный экземпляр класса `GameUiState`. Присвойте его новой неизменяемой переменной `gameUiState`:

```
@Test
fun gameViewModel_Initialization_FirstWordLoaded() {
    val gameUiState = viewModel.uiState.value
}
```

Весь следующий код пишем внутри функции

`gameViewModel_Initialization_FirstWordLoaded()`

```
val unscrambledWord = getUnscrambledWord(gameUiState.currentScrambledWord)
```

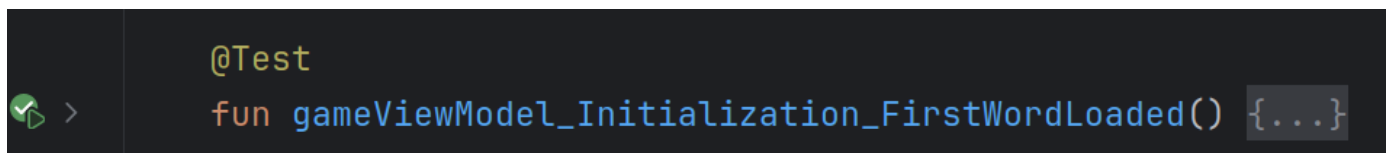
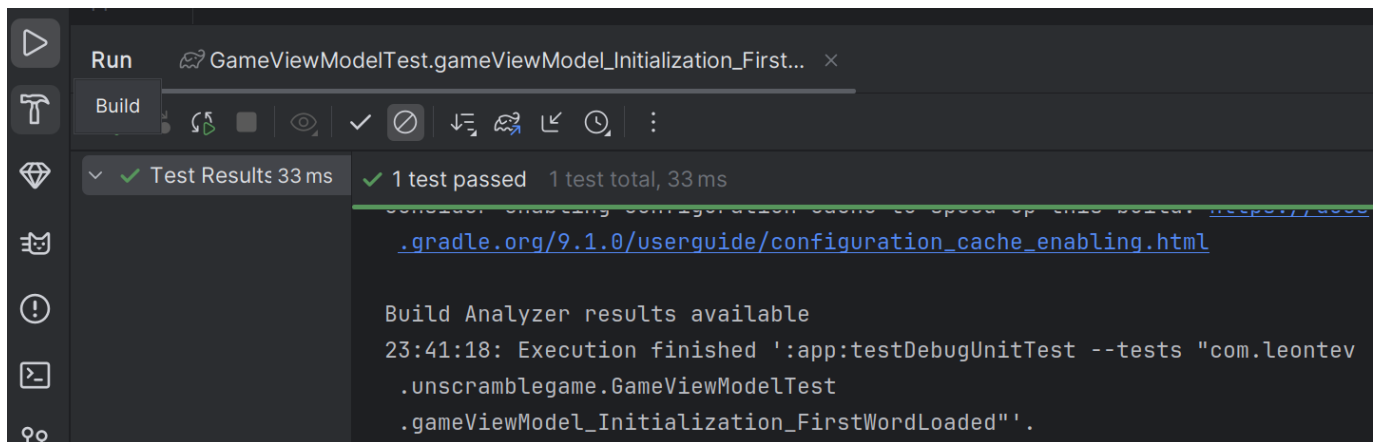
3. Чтобы убедиться, что состояние корректно, добавьте функции `assertTrue()` для проверки, что:

- свойство `currentWordCount` установлено в 1;

- свойство `score` установлено в 0.
- Добавьте функции `assertFalse()` для проверки, что:
 - свойство `isGuessedWordWrong` равно `false`;
 - свойство `isGameOver` равно `false`.

```
assertNotEquals(unscrambledWord, gameUiState.currentScrambledWord)
assertTrue(gameUiState.currentWordCount == 1)
assertTrue(gameUiState.score == 0)
assertFalse(gameUiState.isGameOver)
```

Запустите тест и убедитесь, что он успешно проходит:



Ещё один граничный случай — это тестирование состояния UI после того, как пользователь угадал все слова. Необходимо проверить, что когда пользователь правильно угадывает все слова, выполняется следующее:

- Счёт обновлён корректно;
- Значение свойства `currentGameUiState.currentWordCount` равно значению константы `MAX_NO_OF_WORDS`;
- Значение свойства `currentGameUiState.isGameOver` установлено в `true`.

Выполните следующие шаги, чтобы добавить тест:

- Создайте метод

`gameViewModel_AllWordsGuessed_UiStateUpdatedCorrectly()` и аннотируйте его аннотацией **`@Test`**:

```
@Test
fun gameViewModel_AllWordsGuessed_UiStateUpdatedCorrectly() {...}
```

- Внутри метода создайте переменную `expectedScore` и присвойте ей значение 0:

```
@Test
fun gameViewModel_AllWordsGuessed_UiStateUpdatedCorrectly() {
    var expectedScore = 0
}
```

Весь следующий код пишем внутри функции

`gameViewModel_AllWordsGuessed_UiStateUpdatedCorrectly()`

- Чтобы получить начальное состояние, добавьте переменную `currentGameUiState` и присвойте ей значение свойства `viewModel.uiState.value`:

```
var currentGameUiState = viewModel.uiState.value
```

- Чтобы получить правильное слово, введённое игроком, используйте функцию `getUnscrambledWord()`, которая принимает в качестве аргумента `currentGameUiState.currentScrambledWord` и возвращает расшифрованное слово. Сохраните возвращаемое значение в новой переменной `correctPlayerWord`:

```
var correctPlayerWord = getUnscrambledWord(currentGameUiState.currentScrambledWord)
```

- Чтобы протестировать сценарий, в котором пользователь угадывает все ответы, используйте блок `repeat`, чтобы повторить выполнение методов `viewModel.updateUserGuess()` и `viewModel.checkUserGuess()` `MAX_NO_OF_WORDS` раз:

```
repeat(MAX_NO_OF_WORDS) {
    // ...
}
```

Внутри блока `repeat`:

- увеличивайте значение переменной `expectedScore` на значение константы `SCORE_INCREASE`, чтобы подтвердить, что счёт увеличивается после каждого правильного ответа;

- вызовите метод **viewModel.updateUserGuess()** и передайте в него переменную **correctPlayerWord**;
- вызовите метод **viewModel.checkUserGuess()** для проверки ответа пользователя.
- Обновляйте текущее слово игрока, используя функцию **getUnscrambledWord()**, которая принимает **currentGameUiState.currentScrambledWord** в качестве аргумента и возвращает расшифрованное слово. Сохраните это значение в переменной **correctPlayerWord**.
- Чтобы убедиться, что состояние корректно, добавьте функцию **assertEquals()** для проверки, что значение свойства **currentGameUiState.score** равно значению переменной **expectedScore**:

```
repeat(MAX_NO_OF_WORDS) {
    expectedScore += SCORE_INCREASE
    viewModel.updateUserGuess(correctPlayerWord)
    viewModel.checkUserGuess()
    currentGameUiState = viewModel.uiState.value
    correctPlayerWord = getUnscrambledWord(currentGameUiState
        .currentScrambledWord)
    assertEquals(expectedScore, currentGameUiState.score)
}
```

- Добавьте функцию **assertEquals()**, чтобы проверить, что значение свойства **currentGameUiState.currentWordCount** равно значению константы **MAX_NO_OF_WORDS**, а значение свойства **currentGameUiState.isGameOver** установлено в **true**:

```
repeat(MAX_NO_OF_WORDS) {...}
assertEquals(MAX_NO_OF_WORDS, currentGameUiState.currentWordCount)
assertTrue(currentGameUiState.isGameOver)
```

- Запустите тест и убедитесь, что он успешно проходит:

```
@Test
fun gameViewModel_AllWordsGuessed_UiStateUpdatedCorrectly() {...}
```


Обзор жизненного цикла экземпляров тестов

Если внимательно посмотреть на то, как **viewModel** инициализируется в тесте, можно заметить, что **viewModel** создаётся только один раз, несмотря на то что используется во всех тестах. Следующий фрагмент кода показывает определение свойства **viewModel**:

```
class GameViewModelTest {  
    private val viewModel = GameViewModel()  
  
    @Test  
    fun gameViewModel_Initialization_FirstWordLoaded() {...}
```

Может возникнуть вопрос:

- Означает ли это, что один и тот же экземпляр **viewModel** используется во всех тестах?
- Может ли это вызвать проблемы? Например, что произойдёт, если метод **gameViewModel_Initialization_FirstWordLoaded** выполнится после метода **gameViewModel_CorrectWordGuessed_ScoreUpdatedAndErrorFlagUnset**?

Провалится ли тест инициализации?

Ответ на оба вопроса — **нет**. Методы тестов выполняются изолированно, чтобы избежать неожиданных побочных эффектов из-за изменяемого состояния тестового экземпляра. По умолчанию перед выполнением каждого тестового метода JUnit создаёт новый экземпляр тестового класса.

Поскольку в классе **GameViewModelTest** на данный момент есть четыре тестовых метода, **GameViewModelTest** будет создаваться четыре раза. Каждый экземпляр имеет собственную копию свойства **viewModel**. Следовательно, порядок выполнения тестов не имеет значения.

Скриншоты для отчёта



Сделайте скриншот:

- Результаты выполнения тестов (все должны быть зелёными)
- Сохраните как: **step8_tests_passed_FIO.png**

Выполните в терминале:

git add .

git commit -m "Step 8: Add unit tests for GameViewModel"

git push

Самостоятельная работа

Задание: Создание README.md

Создайте файл **README.md** в корне проекта, который должен содержать:

1. Название проекта
2. Краткое описание игры
3. Технологии (Kotlin, Jetpack Compose, ViewModel, StateFlow)
4. Функциональность приложения (список возможностей)

Ответьте на следующие вопросы:

Контрольные вопросы

1. Что такое ViewModel и зачем он нужен?

- Объясните своими словами, какую проблему решает ViewModel
- Что происходит с данными при повороте экрана без ViewModel?

2. В чём разница между MutableStateFlow и StateFlow?

- Зачем создавать два поля: `_uiState` и `uiState`?
- Что такое “backing property pattern”?

3. Почему в ViewModel нельзя хранить Context или View?

- Что может произойти, если сохранить ссылку на Activity в ViewModel?

4. Чем StateFlow отличается от remember и rememberSaveable?

- Когда лучше использовать каждый подход?

5. Что такое data class в Kotlin и зачем он используется для UI State?

- Какие методы автоматически генерируются для data class?
- Зачем нужна функция `copy()`?

6. Как работает collectAsState() в Compose?

- Почему UI автоматически обновляется при изменении StateFlow?

7. Зачем разделять код на пакеты data и ui?

- Какие принципы архитектуры это поддерживает?

8. Что происходит с ViewModel, когда приложение полностью закрывается (process death)?

- Сохраняются ли данные в этом случае?
- Какие альтернативы есть для долговременного хранения?

После выполните в терминале:

git add README.md

git commit -m "Add project README"

git push